Towards Utilizing Cached Context for Faster and Smarter Code Agents

Frank Chen

CMU-CS-25-139

August 2025

Computer Science Department
School of Computer Science
Carnegie Mellon University
Pittsburgh, PA 15213

Thesis Committee
Prof. Rashmi K. Vinayak, Chair
Prof. Zhihao Jia

Submitted in partial fulfillment of the requirements for the degree of Master of Science.

Keywords: LLM, Agent, Cache, Retrieval Augmented Generation, Software Engineering

Abstract

Recent advances in Large Language Models (LLMs) have enabled the development of coding agents that can autonomously perform tasks such as code generation, debugging, patch validation, etc. Industry-proprietary systems (e.g., Colab AI, Claude Code, Cursor Agent) and open-source frameworks (e.g., Gemini Cli, SWE-Agent, AutoCodeRover, Agentless) have demonstrated both practicality and popularity in real-world software engineering workflows. Despite these successes, existing agentic frameworks face two common challenges: providing accurate and complete context information and ensuring low-latency patch generation under heavy workloads. Although prior work has proposed partial solutions addressing either context retrieval or latency, little attention has been paid to the joint optimization of both aspects. Ideally, joint optimization should enhance both performance and speed without incurring additional cost, which is particularly critical in modern fast-paced, iterative software engineering environments. This thesis investigates integrating existing state-of-the-art approaches to these challenges, specifically RepoGraph for smart context retrieval on the frontend and CacheBlend for fast inference on the backend. To verify feasibility of an integration, we then evaluate performance of the frontend across multiple LLMs under realistic code-agent scenarios, and measure the latency improvement of the backend against systems such as vLLM and SGLang on trace generated by frontend under the same benchmark. The results highlight the trade-offs between cost, efficiency, and performance and argue for the necessity of integrated solutions that achieve balance between the factors mentioned. Finally, we suggest a preliminary design for an end-to-end system that combines the benefits of RepoGraph and CacheBlend via a simple adapter module, along with optimizations in the original algorithms. Overall, the findings suggest promising directions toward building a robust coding agent that is both fast and high-performing.

Acknowledgments

Contents

List of Figures

Chapter 1

Introduction

<TODO: a bit more background on agents?>

Recent studies on AI coding agents have proven the efficacy of using Large Language Models (LLMs) to handle coding tasks such as code generation and debugging. To apply LLMs in daily software engineering scenarios, both industry and academia have proposed many agentic frameworks. Industry-proprietary examples of agentic frameworks include Google's Colab AI [SKS25], Claude Code [Ant25], and Cursor Agent [LIB25]. There is a continuous and rapid growth of user bases for the above frameworks, reflecting their perceived utility in real-world software engineering tasks. This implies increasing demand for production-ready AI assistants that integrate seamlessly into existing development workflows. Besides those proprietorial frameworks, there are also many open source implementations from both academia and industry. Examples of these open-source agentic frameworks include Gemini Cli [MS25], SWE-Agent [Yan+24], AutoCodeRover [Zha+24c], Agentless [Xia+24], etc. These frameworks enable existing LLMs to achieve human-level performance when prompted with clear instructions and sufficiently rich context. Despite these advances, current agentic systems still face two major challenges : 1. How to provide accurate and complete context information to the LLM for the best performance / code quality (later referred to as **Challenge 1**) [Ouy+25]; 2. How to ensure fast patch generation when the LLM is fed with a large number of requests and/or large number of tokens during codebase exploration (later referred to as **Challenge 2**).

More recently, there have been works that address either Challenge 1 or Challenge 2[Yixuan: Briefly describes what existing work does. I assume there will be a few papers that address challenge 1 with some context retrial algorithm, and challenge 2 with prefix caching] [Frank: There are many papers relevant to just repository level context retrieval, but they are for inline code completion/copilot instead of agents. Should I include those and say they are relevant?][Yixuan: Yes, but at a very high level, i.e. a few words / a short sentence]. However, no existing system attempts to tackle both challenges at once. [Yixuan: Need to motivate here why we want to perform a joint optimization. Show with some examples.] There are many reasons why a joint optimization is highly desirable, the most apparent being cost. <TODO: find relevant examples, I think Swebench/sweagent paper has data on cost per issue>. For a complex code repository and an intricate coding

task , it often takes a significantly long time for the agent to generate a patch, even when the patch is unlikely to resolve the issue given the complexity. And if the patch is deemed problematic, the consumed time and token length all contribute to the cumulative engineering expense. Despite the benefits, joint optimization remains largely unexplored given the lack of understanding in how to tailor front-end and back-end optimizations to work together in an agentic setting. [Yixuan: How is this related to why people don't do joint optimization?]

This thesis project aims to fill the gap in understanding: what are the solutions used by existing frameworks to address the challenges, how these solutions affect the performance, and what trade-offs need to be considered for joint optimization of context retrieval and patch generation latency. Our results and analysis illustrate the possibility as well as difficulty of integrating such solutions to step towards a fast yet performant code LLM-based agentic framework. The paper is organized as follows:

- In Chapter 2, we provide background on the two key challenges: providing accurate and complete context information to the LLM (Challenge 1) and ensuring fast patch generation under heavy workloads (Challenge 2). We survey existing approaches and state-of-the-art solutions for each, specifically RepoGraph [Ouy+25] as a context retrieval algorithm addressing Challenge 1, and CacheBlend [Yao+25] as a caching algorithm targeting Challenge 2. We then motivate the need for joint optimization of both challenges.
- In Chapter 3, we present the design of our end-to-end system that integrates RepoGraph and CacheBlend via the *Cached Context* concept, enabling position-independent KV cache reuse on context-rich agentic prompts. We also describe the system architecture, adapter module, and implementation optimizations.
- In Chapter 4, we evaluate the system along two axes: frontend context retrieval performance across multiple LLM backends (including the impact of CacheBlend on output quality), and backend KV cache reuse latency under agentic workloads.

Chapter 2

Background

This chapter provides the technical background for the two key challenges facing modern coding agents. Section 2.1 discusses the problem of providing accurate and complete context to LLMs operating on large code repositories, along with RepoGraph as a state-of-the-art context retrieval algorithm. Section 2.2 addresses the latency bottleneck caused by redundant computation in agentic workflows, presenting CacheBlend as an approach to position-independent KV cache reuse. Finally, Section 2.3 motivates the need for a joint optimization that addresses both challenges in a unified system.

2.1 Repository-Aware Context Retrieval

Modern code LLM agents have shown impressive capabilities, but most still struggle with incomplete or irrelevant context when working on large repositories. Many frameworks, from early tool-using agents to recent systems like SWE-Agent [Yan+24], AutoCodeRover [Zha+24c], and Agentless [Xia+24], often feed the model only limited snippets of a codebase or generic summaries. Conventional agents often are able to interact with the entire codebase via searching keywords, files, and directories, in addition to some "scrolling" mechanism for file viewing, that is, having a constant-sized context window that can be moved up or down to have a view on an arbitrary code snippet of any file. SWE-agent's implementation, for example, consists of three commands: `goto`, `scroll_up`, and `scroll_down`, allowing the agent to move the context window to any line of the target file [Yan+24]. However, this does not prevent them from getting stuck in a local optimum [Ouy+25] [Yixuan: what does "local optimum" refer to, better be specific]. What this means is that the agent consistently has a limited view on the repository, resulting in possible false assumptions on code structure and implementation details. This lack of essential context can lead to unstable performance and failures in code generation or bug-fixing (we refer to this as **Challenge 1**). For example, providing an agent with only a function implementation but not definitions of its referenced symbols (classes, functions, etc.) can omit crucial information and hurt the quality of patches[Yixuan: A figure to illustrate this example would be good.] <TODO: Add something like a referenced function

is async but agent is unaware>. Conversely, naively supplying too much code (e.g., multiple flat files or irrelevant documentation/config) can overwhelm the model with irrelevant details, interfering with its reasoning [Zha+24a; Zha+24b]. Previous studies have noted the challenge of balancing concise yet sufficient context given the finite context window of LLMs [Ouy+25].

More broadly, code repositories present unique challenges for LLM-based agents due to their scale and interconnectivity. Real-world codebases often contain hundreds of thousands of lines of code spread across many files. This makes it infeasible to simply dump the entire repository into the LLM's context window. Instead, an agent must *localize* the relevant portions of code for a given task. Traditional program analysis techniques, such as dependency graphs and call graphs, have long been used to understand large codebases, and recent LLM agents are beginning to incorporate such ideas. Early coding assistants relied on heuristic searches (e.g. grepping for error messages or function names) and would open files one by one in a Localize-then-Edit paradigm. More modern agents improve on this by indexing repository knowledge: for example, SWE-Search [Ant+25] integrate a Monte Carlo Tree Search to explore the complex solution space, and CodePlan [Bai+23] tracks incremental code changes with dependency analysis.

Despite these advancements, providing just the right context remains difficult. A key issue is that repository-level issue solving often requires following chains of references across multiple files. If any link in the chain is missing from the prompt, the model may fail to resolve the problem. Conversely, adding too much code can dilute the context. CoSIL [Jia+25] [Yixuan: Use this to support the sentence "Conversely, naively supplying too much code ..." in Sec. 2.1.] highlights this tension: it points out that existing methods struggle to give concise yet effective context coverage . CoSIL's solution is to dynamically construct a module-level call graph during the agent's search, expanding the graph by following function calls and pruning irrelevant branches.

RepoGraph [Ouy+25] takes a complementary approach by pre-building a complete repository graph before the agent starts working, offering a state-of-the-art solution to Challenge 1 by explicitly modeling the structure and dependencies of the repository. It constructs a comprehensive *repository-level code graph* that captures relationships between code elements across different files. Each class, method, or function is a node, and the edges connect definitions to their usage or dependency. During a setup phase, RepoGraph parses all code files (using `tree-sitter`) to identify definitions (classes, functions, variables) and their references, filters out non-code files, and stores the resulting graph for fast querying. When the agent needs context for a particular function or error, it can query this graph to retrieve all related definitions and usage chains. In essence, RepoGraph provides an oracle that, given a symbol or filename, returns a subgraph containing all immediately relevant symbols. <TODO: include a graphic example here>. By following these links, the agent is much less likely to overlook a relevant piece of code compared to linear search or limited file context. This approach has proven effective with GPT4/4o or Claude-3.5-Sonnet as backend: when plugged into Agentless or SWE-Agent, RepoGraph consistently boosted their success rates [Ouy+25]. This makes RepoGraph ideal as the foundation of our frontend context retrieval algorithm, whose specific usage will be discussed in closer detail in Chapter 3.

<TODO: eval related insights, especially why comprehensive eval with diff APIs is important>

2.2 Position-Independent KV Cache Reuse

Besides quality, the other major limitation of current code LLM agents is latency (**Challenge 2**). Many agentic frameworks send a large number of sequential requests to the LLM as it undergoes localization. This iterative process is significantly time-consuming, especially when the prompt in each request includes bulk context information as well as template tokens, and the model re-processes it from scratch every time. For instance, an agent might retrieve a large snippet of code from RepoGraph, feed it into the model to propose a fix, then in a subsequent step feed much of the same snippet again while testing or refining the fix. When no optimizations are prepared for this case, each of those calls recomputes attention over the repeated context tokens, undergoing the same KV cache prefill stage and wasting computation. This redundant computation is a key inefficiency that is shared by many agentic workflows.

At the core of this inefficiency is the key-value (KV) cache in transformer models, which stores intermediate attention states of processed tokens so that each new token need not recompute attention over the entire sequence. However, this cache normally resets between separate API calls. To reuse computation across calls, several optimized LLM serving engines have introduced caching and scheduling strategies. vLLM [Kwo+23] improves throughput by batched execution and paged memory management of the KV cache, treating it as pageable memory. It can serve multiple generation requests in parallel and merge requests with identical prefixes so that the shared prefix is computed only once. However, even vLLM only reuses prefix computations when those prefixes exactly align from the start of the prompt. In an agent scenario, calls often share partial contexts that appear after some unique tokens, leading to low cache hit rates. SGLang [Zhe+24] introduces RadixAttention, storing caches in a radix tree keyed by prefixes and scheduling requests to maximize cache hits, though it too relies on exact prefix matching. Another approach, Prompt Cache [Gim+24], allows explicitly marking reusable prompt segments and precomputing their KV states. While this yields substantial latency reductions on long-document QA tasks, it requires manual template setup and cannot adapt if templates change.

Cache optimization for LLM inference has recently taken a leap forward with CacheBlend [Yao+25]. CacheBlend's key innovation is enabling *position-independent* reuse of KV caches. It treats the prompt as composed of chunks, checks for each chunk if it has stored KV state, and when reassembling chunks selectively recomputes a small subset of tokens—specifically HKVD (High-KV-Deviation) tokens—to mitigate attention deviation and update positional encodings to reflect shifts in token positions, restoring the impact of cross-attention on the forward-attention matrix. The observation that HKVD tokens tend to remain deviated through different layers also makes it possible for efficient selection and KV recomputation of such tokens. This "blending" enables much higher cache hit rates on retrieval workflows, yielding 2–3× reduction in time-to-first-token and 3–5×

throughput improvements, all without degrading output quality. CacheBlend was proved effective in multi-round QA scenarios through datasets such as 2WikiMQA and Musique [Ho+20; Tri+22] <TODO: maybe expand a bit on previous evaluations>. However, there remains a gap in understanding how CacheBlend performs in real-world agentic settings, where prompts can consist of long code context and complex repository structure information. Moreover, the original implementation assumes contiguous prefix alignment for cache lookup and allocates memory objects one at a time per cache chunk, which introduces scalability limitations under the high concurrency and variable prompt lengths characteristic of agentic workloads. In this work, we extend CacheBlend with non-contiguous cache hit support and enhanced memory allocation to address these limitations (see Chapter 3), and evaluate how the augmented system reduces latency under agentic use cases by integrating it into a RepoGraph-augmented agent scenario and measuring end-to-end latency against baseline systems like vLLM (with and without caching enabled).

2.3 Motivation for Joint Optimization

Having identified **Challenge 1** (poor patch quality due to incomplete context) and **Challenge 2** (slow generation due to long context) as two fundamental limitations of modern agentic systems, it is natural to consider designing an end-to-end system that aims at tackling both challenges, given the combined performance, latency, and cost benefits of a joint optimization. In the preceding sections, we surveyed state-of-the-art solutions for each respective challenge: RepoGraph [Ouy+25] for context retrieval and CacheBlend [Yao+25] for inference latency. RepoGraph extracts repository-level context via pre-generating a structure tree containing symbol locations and a code graph containing call relations. CacheBlend enables position-independent KV cache full reuse by selectively recomputing a small subset of high-KV-deviation tokens to restore cross-attention. In our integrated system, the aim is to combine the performance and latency advantages of each while mitigating the degradation of accuracy due to partially recomputed KV cache. The core concept that allows this is *Cached Context*, which is a simple yet effective idea of chunking context blocks in prompts generated in the agentic workflow, such that the back-end KV cache reuse scheme (CacheBlend in our case) is able to exploit reuse opportunities more frequently. The reason why this is particularly effective in agentic settings is primarily because of the modular and repetitive nature of code agent context, which mostly consists of repository structure related info and function or class definition code blocks. Prompt templates, often containing format requirements and task-specific instructions, also will have tokens that are repeated upon every new request sent to the LLM. *Cached Context* allows capturing these additional reuse opportunities to further speed up inference without loss of accuracy.

Chapter 3

System Design

Our end-to-end system design centers around the concept of *Cached Context*, which provides a simple yet effective approach to maximizing KV cache reuse opportunities in agentic workflows. The key idea is to chunk each prompt at a per-context-object granularity, where a context object corresponds to a code snippet containing a symbol definition, a repository structure tree, or other semantically coherent unit of context.

3.1 Cached Context

To illustrate the advantage of *Cached Context* over traditional prefix caching, consider the following example. Suppose the agent constructs a prompt after context retrieval, consisting of four parts: system instructions, repository structure, issue description, and relevant code snippet. A naive chunking strategy would treat this prompt as a single unit or divide it into four coarse blocks. With *Cached Context*, we are able to further decompose the context blocks into finer-grainer blocks. The "relevant code snippets" block, for example, might contain definitions for multiple symbols retrieved by RepoGraph, where each symbol definition is now its own cache chunk.

Now consider a subsequent prompt in the same workflow <TODO: visualize, can just tweak the present slides figure a little>, but with slightly different localization and set of code snippets, while most symbols and other blocks remain the same. Under prefix caching, unless the overlapping content appears from the start of the prompt, the cache cannot be reused. But under *Cached Context*, each overlapping symbol definition is a cache hit regardless of its position. This position-independent reuse is particularly valuable in agentic workflows where the order and combination of context elements frequently varies.

3.2 System Architecture

Figure 3.1 illustrates the overall architecture of our end-to-end system. The design prioritizes modularity and extensibility, allowing components to be developed and tested independently. Design details per module are as follows:

3.2.1 Frontend: RepoGraph + Agentless

We deploy RepoGraph as the frontend context retrieval algorithm and integrate it with the Agentless procedural framework. Agentless follows a three-step routine:

1. **Localize**: Given an issue description, identify the files and code regions most likely to require modification.
2. **Repair**: Generate candidate patches for the localized regions.
3. **Rerank**: Evaluate and rank the candidate patches, selecting the most promising one for submission.

RepoGraph enhances the localization phase by providing repository-aware context. Rather than relying solely on file-level search, Agentless can query RepoGraph's code graph to retrieve symbol definitions, call chains, and structural information. This integration ensures that the LLM receives semantically complete context—for example, when localizing a bug in a function, the model also receives definitions of all symbols referenced by that function.

To enable cached context, we modify Agentless's prompt generation routine to explicit context objects, therefore exposing chunk boundaries to the backend adapter without affecting the semantic content of the prompt.

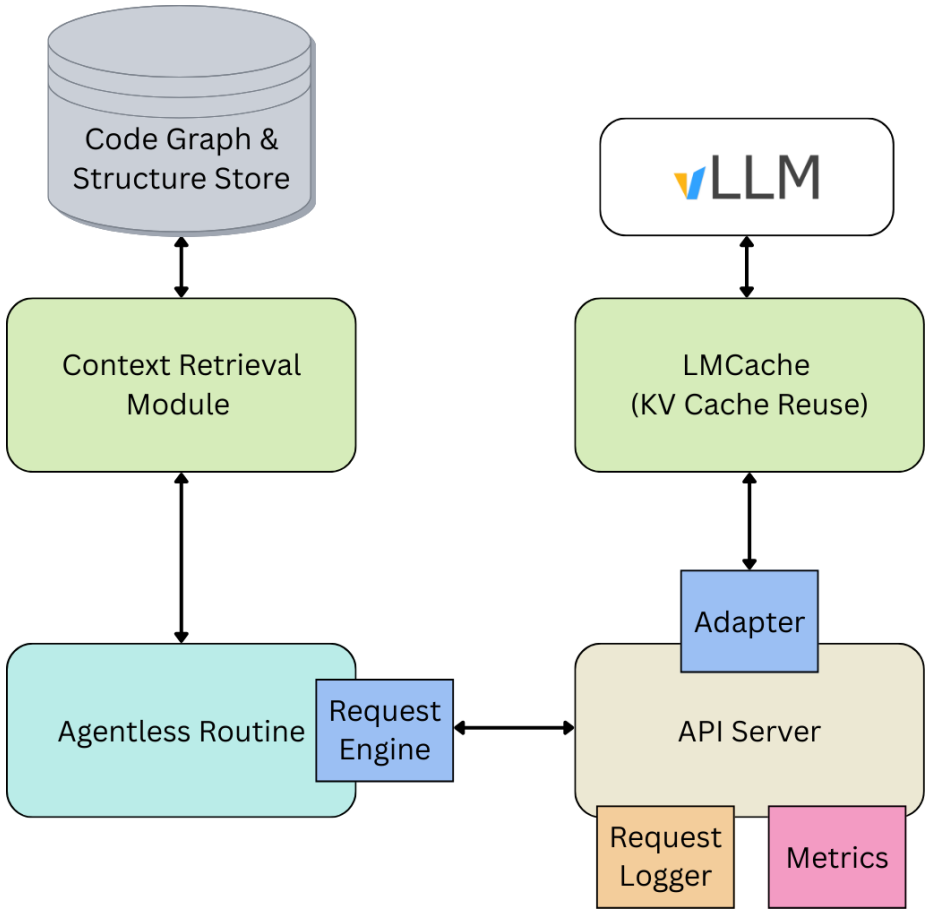

Figure 3.1: End-to-end system architecture

3.2.2 Backend: API Server with CacheBlend

The backend consists of an API server that serves as the entry point for frontend requests. The server hosts the LLM inference engine (vLLM) with LMCache integration, which implements the CacheBlend algorithm for position-independent KV cache reuse. When a request arrives, the server:

1. Parses the chat messages from the request.
2. Passes the messages through the adapter module (described below) for tokenization and chunk extraction.
3. Forwards the processed prompt to the LLM engine with CacheBlend enabled.
4. Returns the generated response to the client.

This modular design ensures that any compatible frontend can benefit from cached context without modification beyond annotating context object boundaries.

3.2.3 Adapter Module

The adapter module serves as the bridge between the frontend prompt format and the backend cache system. Its responsibilities include:

1. **Chunk extraction**: Identifying and retrieving individual context objects.
2. **Tokenization**: Encoding each chunk using the model's tokenizer, as well as the correct handling of models with custom tokenization schemes (e.g. Devstral).
3. **Chunk stitching**: Reassembling the tokenized chunks to produce the final prompt token sequence.

The adapter's design allows for flexible chunking granularity based on frontend prompt template and user preference. For example, if chunking is based on context per symbol, the frontend adds a delimiter after each code snippet fetched during code graph exploration. If coarser granularity is desired (e.g., per context block), delimiters can be placed only between major sections of the prompt. This provides robustness against various workload patterns and different optimal chunk sizes.

3.3 Local Store for Precomputed Data

Computing the code graph and repository structure from scratch for every task instance incurs significant latency overhead. RepoGraph's graph construction involves parsing all source files using tree-sitter, identifying symbol definitions and references, and building the dependency graph. For large repositories, this process can take several minutes.

To address this, we introduce a local store that caches these large data structures. The store is keyed by repository ID and commit hash, ensuring that cached data remain valid across tasks targeting the same codebase state. This optimization is particularly valuable when processing multiple issues against the same repository (common in benchmark evaluations like SWE-bench) or when the same repository is revisited across different agent sessions.

3.4 Request Logging and Replay

To support debugging, profiling, and reproducible experimentation, the API server includes a request logging module. During a server session, all incoming requests and their corresponding responses are captured and stored as a local trace file.

This transparency enables inspection of which stage during the Agentless routine may have caused a performance bottleneck. For example, if localization prompts consistently exhibit low cache hit rates, the prompt template can be adjusted to improve context alignment across requests.

In addition, the server supports a **replay** functionality that parses requests from any trace file and simulates the workload internally. This feature enables:

- Evaluation on captured traces from real agentic workloads without re-running the full agent pipeline.
- Modularized testing through synthetic small traces that exercise specific caching scenarios.
- A/B comparison of different caching configurations on identical workloads.

Both capabilities are desirable for developing a production-ready system, as they allow performance analysis and optimization to proceed independently of the agent's decision-making logic.

3.5 Implementation

Beyond the core architecture, several implementation optimizations improve the system's robustness and performance.

3.5.1 Context Truncation

For complex coding tasks involving multiple files and nested dependencies, it is possible for the frontend to exceed the model's token limit when constructing a prompt. Originally, prompts were naively truncated at the end when the token limit was reached. This approach frequently results in important instructions (such as output format requirements or validation criteria) being omitted from the prompt. Even when the LLM receives sufficient context to understand the problem, it may underperform due to missing instructions.

<TODO: adapt visualization from pres slides>

Our solution leverages the model's tokenizer to calculate the token length of each context block and the prompt template. Then before constructing the prompt, the last context block is truncated to exactly fit the remaining token budget, so that the critical pieces on the template will stay intact and instructions will be passed to the LLM as desired. This improves the reliability of the generated patches when the context is long.

3.5.2 Non-Contiguous Cache Hit Support

The original CacheBlend implementation requires contiguous prefix alignment for cache lookup: if the prompt begins with tokens already cached by the serving engine (e.g., vLLM's prefix cache), only the suffix following that prefix is eligible for segment-level cache matching. In an agentic workflow, however, the set of context objects retrieved across successive requests frequently overlaps only partially and in non-consecutive positions. For example, if prompt A includes context chunks $\{C_1, C_3, C_4\}$ and a subsequent prompt B includes $\{C_2, C_3, C_4\}$, the original scheme would miss the reusable suffix $\{C_3, C_4\}$ because C_2 differs from C_1 and breaks prefix contiguity.

To address this, we modify the segment token database to hash each chunk independently after the masked prefix. Given a mask of the form `F...FT...T`, where leading `F` entries denote tokens already handled by the engine's prefix cache, each segment among the remaining `T` entries is hashed and looked up in the cache store separately. This enables non-contiguous cache hits: any subset of chunks that appeared in a prior request can be reused regardless of its position in the current prompt. The modification preserves backward compatibility—prompts with contiguous overlap behave identically to before—while substantially increasing hit rates in multi-turn agentic workloads where context objects are reordered or partially substituted between requests.

3.5.3 Enhanced Memory Allocation and Locking

Scaling CacheBlend to agentic workloads with high request concurrency and large prompt lengths exposed two bottlenecks in the storage backend's memory management path.

Batched varied-shape allocation. Each cache chunk corresponds to a memory object whose shape depends on the chunk's token count. The original implementation allocated these objects one at a time, acquiring and releasing the CPU storage lock for each chunk. Under high concurrency, this per-chunk locking becomes a serialization bottleneck: a single store operation for a prompt comprising N chunks requires N lock acquisitions. We introduce a `batched_allocate_varied` method that accepts a list of shapes in one call, acquires the lock once, performs all allocations (with bulk eviction if capacity is exceeded), and releases the lock. This reduces lock contention by a factor of N per request and avoids partial-allocation states where some chunks succeed while others fail due to concurrent eviction.

Bounded eviction with deadlock prevention. When the cache is full, the allocator must evict existing entries to make room. In the original design, the eviction loop retried indefinitely until sufficient space was freed. Under heavy concurrent load, this could deadlock: if all cache entries were pinned by in-flight lookups on other threads, no entry was eligible for eviction, and the allocator would spin indefinitely while holding the storage lock. We bound the eviction loop to a configurable maximum number of retries. If the limit is reached, the allocation fails gracefully and the request falls back to full prefill computation, avoiding system-wide stalls.

3.5.4 Per-Request Profiling Infrastructure

To enable fine-grained latency analysis of the cache engine under realistic workloads, we implement a per-request profiling module that instruments the critical path through the adapter, cache engine, storage backend, and GPU connector. The profiler is organized around two abstractions: *operations*, which correspond to logical units of work (e.g., a single request's retrieve pass), and *phases*, which subdivide an operation into timed intervals (e.g., token database lookup, per-layer storage retrieval, per-layer GPU onload).

The profiler uses thread-local storage to maintain a per-thread operation stack, ensuring that timing records from tensor-parallel workers do not bleed across threads. Each phase is recorded with its wall-clock duration, parent operation identifier, and thread ID. On completion, the profiler emits per-operation summaries in JSONL format, enabling post-hoc aggregation and visualization.

Figure 3.2 shows an example timeline produced by the profiler during a CacheBlend-enabled serving session. Each point represents a single profiled operation, plotted by wall-clock time (x-axis) against its measured latency (y-axis), and color-coded by operation type. The `retrieve_layer` operations (orange) constitute the bulk of per-step cache engine overhead, typically clustering between 25–75 ms with occasional spikes above 100 ms. The `store_layer` operations (green) show a similar range, corresponding to the cost of persisting newly computed KV states. The `adapter.start_load_kv` markers (red, near the baseline) indicate the lightweight entry point into the cache path. This visualization enables rapid identification of latency outliers and temporal patterns that would be invisible in aggregate statistics.

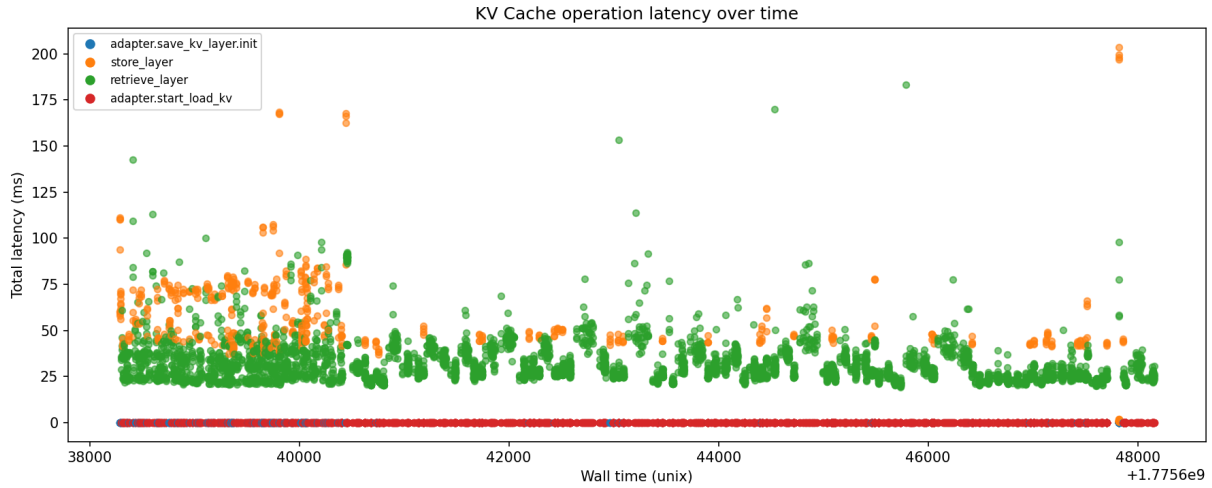

Figure 3.2: Example profiler output: per-operation latency timeline during a CacheBlend-enabled serving session (chunk lower bound 128 tokens). Each point is a single profiled operation, color-coded by type. The clustering of `retrieve_layer` (orange) at 25–75 ms reflects the fixed per-step overhead analyzed in Section 4.2.3.

This infrastructure is critical for diagnosing performance anomalies that are invisible at the aggregate level. As discussed in Section 4.2.3, the profiler's per-request phase break-

down revealed the root cause of high speedup variance among small requests—an insight that would not have been possible from end-to-end TTFT measurements alone.

Chapter 4

Evaluations

To evaluate the effectiveness of our system, we conduct two sets of evaluations:

- Performance evaluation of RepoGraph, the frontend context retrieval algorithm, across models of different sizes and specialties. This assesses the consistency, robustness, and portability of the algorithm when under resource constraints and/or the backend model's reasoning capability is limited. We additionally evaluate the impact of CacheBlend's KV cache reuse on output quality.
- Latency evaluation for the enhanced CacheBlend backend, specifically under agentic workloads generated by the RepoGraph frontend. This evaluation uses the per-request profiling infrastructure (Section 3.5.4) to measure TTFT speedup as a function of cache hit ratio and prompt length, and investigates the interaction between vLLM's chunked-prefill scheduler and the cache engine's per-step overhead. This differentiates from the original work's evaluations with QA and text-based RAG workloads, and is critical for understanding the applicability of the caching scheme in agentic settings.

4.1 Performance Evaluation for Context Retrieval

4.1.1 Setup

The evaluation of RepoGraph largely follows the methods of the original paper, where it is utilized as a plug-in component on top of an existing agentic framework. For the purpose of our study, we chose the Agentless procedural framework. The advantage of integrating a procedural framework in lieu of a traditional agent is simplicity and comprehensibility. In the case of Agentless, it deploys a three-phase process of localization, repair, and patch validation [Xia+24]. Intermediate results are transparent between each step, where empty or error responses are immediately identifiable. This allows for finding bottlenecks and comparing performance against various models on step granularity.

Our goal is to verify the consistency of the integrated agent's performance across LLM backends of different parameter sizes and architectures. For this purpose, we conducted

experiments for RepoGraph + Agentless using a set of models with different configurations (for details see Models).

Metric

We adopt resolve rate following the original work as our primary evaluation metric. An instance is considered resolved if: 1. The generated patch is successfully applied to the repository, and 2. The postfix code passes all test suites.

This metric provides a direct measure of end-to-end task success, capturing not only whether a patch is syntactically correct but also whether it integrates cleanly into the repository and satisfies the intended functionality. Compared to intermediate measures such as code similarity or partial compilation success, the resolve rate reflects the practical utility of the generated patches and therefore serves as a good indicator of real-world performance.

Datasets

For a comprehensive yet time and cost efficient evaluation, we run our experiments in SWE-bench Verified-mini [Mar25], which is a subset of SWE-bench Verified[Cho+24] that is selected via k-means clustering and linear programming, such that the marginal distributions of important scores are preserved. This is a smaller dataset than SWE-bench Lite[Yan+24], yet high-quality and well-specified, and is capable of reflecting an agent's performance across a spectrum of task difficulties.

Models

To ensure consistency, robustness, and portability of RepoGraph as our repository-aware context retrieval algorithm, it is important to evaluate it against multiple model backends of varying architectures and parameter scales. Different models exhibit distinct behaviors in terms of tokenization, embedding spaces, and context utilization. A retrieval method that aligns well with one tokenizer or attention mechanism may under-perform when paired with another, which makes cross-model testing critical for avoiding over-specialization.

In addition, models differ in how they exploit the retrieved context. More powerful models may compensate for imperfect retrieval given stronger reasoning and synthesis capabilities, whereas smaller models are more sensitive to accuracy and relevance of the retrieved context. Thus, evaluating across a spectrum of models provides insight into how much of RepoGraph's performance benefits stem from the context retrieval strategy itself. This also allows us to characterize situations where retrieval quality is most impactful, particularly in cost-constrained settings where smaller models are attractive.

Finally, cross-backend evaluation provides practical benefits for deployment. Since real-world systems often migrate between providers (e.g., OpenAI, Anthropic, Mistral), retrieval algorithms must remain portable to ensure stable performance under backend substitution. For our experiments, we therefore consider a representative set of both proprietary and open-source models, including **GPT-4o, o4-mini, Claude 3.5 Sonnet, and Devstral Small**, covering a range of architectures, parameter counts, and costs.

4.1.2 Results

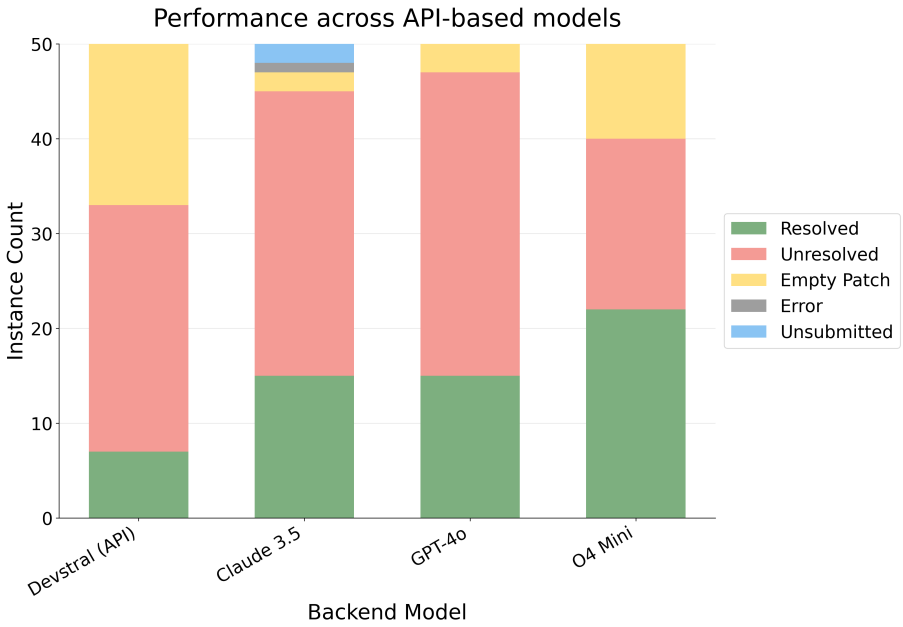

Figure 4.1: Full results distributions for SWE-bench Verified-mini evaluation runs across different models.

To further assess whether CacheBlend's KV cache reuse affects output quality, we run an additional evaluation using Devstral Small hosted on vLLM with LMCache (CacheBlend enabled) on the same SWE-bench Verified-mini dataset. We compare different blending granularities against the baseline (no blending) to verify that position-independent cache reuse does not degrade patch quality.

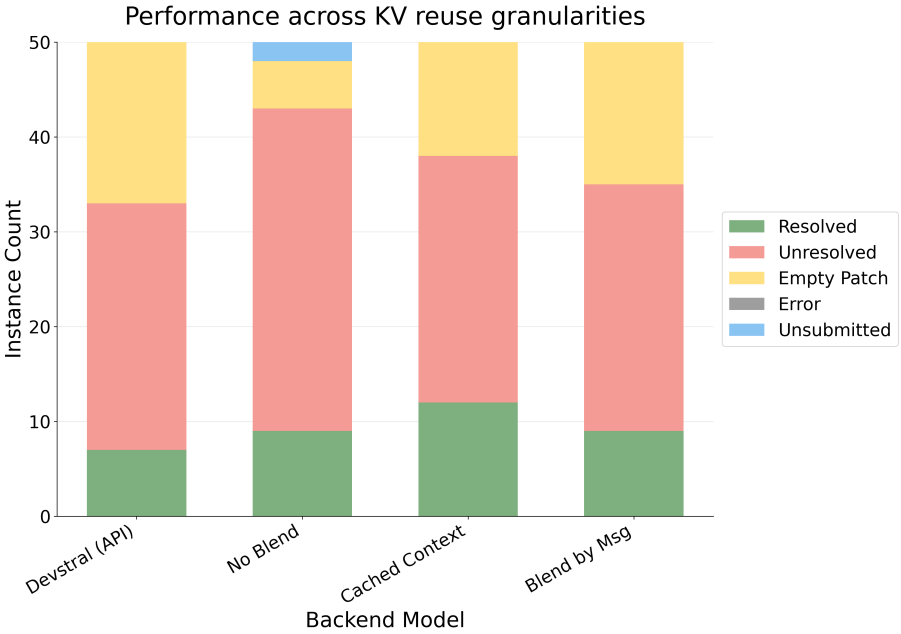

Figure 4.2: Results distributions for SWE-bench Verified-mini evaluation runs across different blending methods, served model is Devstral Small.

4.2 Latency Evaluation for KV Reuse

To assess the latency benefits of CacheBlend under agentic workloads, we measure the time-to-first-token (TTFT) speedup achieved by position-independent KV cache reuse on traces generated by the RepoGraph frontend.

4.2.1 Setup

We evaluate the latency characteristics of CacheBlend by replaying agentic traces generated by the RepoGraph + Agentless frontend against vLLM with and without LMCache integration. All experiments use Devstral Small served with 4-way tensor parallelism. For the blend condition, LMCache is integrated with CacheBlend enabled at a minimum chunk size of 128 tokens. For the baseline condition, the same model is served without LMCache, using only vLLM's native prefix caching. Both conditions process the identical trace via the replay mechanism described in Section 3.4.

Metric

We adopt TTFT as our primary latency metric, defined as the elapsed time from request arrival to the emission of the first output token. TTFT directly captures the prefill-phase cost, which is the phase that cache reuse is designed to accelerate. For each request, we compute the *TTFT speedup* as the ratio of baseline TTFT (no blending, no LMCache) to blend-mode TTFT. A speedup greater than 1.0 indicates that cache reuse reduced prefill latency; a value below 1.0 indicates overhead-induced regression.

Datasets

We replay the full set of prompts produced by RepoGraph + Agentless on SWE-bench Verified-mini [Mar25]. The resulting trace comprises 537 requests spanning a wide range of prompt lengths (119 to over 100,000 tokens) and cache hit ratios (0% to 99.2%), providing coverage across the operating regimes most relevant to agentic use.

Model

All experiments use Devstral Small, consistent with the model used in the performance evaluation (Section 4.1).

Configurations

We compare the following two configurations:

- **Baseline — vLLM (prefix caching)**: vLLM with its built-in prefix caching enabled, which reuses KV states only for exact prefix matches. No LMCache integration.

- **Experimental — vLLM + LMCache (CacheBlend)**: vLLM with LMCache integration and CacheBlend enabled, allowing position-independent KV cache reuse across non-contiguous segments.

4.2.2 Results

Speedup as a Function of Cache Hit Ratio and Prompt Length

Figure 4.3 presents the central result of our latency evaluation: a 3D surface plot of TTFT speedup as a function of cache hit ratio and total prompt tokens, with IQR-based outlier removal (3.0× threshold) applied to remove anomalous data points. The fitted plane

$$\text{speedup} = 1.479 \times \text{hit\_ratio} + 1.09 \times 10^{-6} \times \text{total\_tokens} + 0.427$$

achieves $R^2 = 0.610$ ($n = 532$), indicating that cache hit ratio is the dominant predictor of speedup while total token count has negligible independent effect. The breakeven point—where CacheBlend's overhead is exactly offset by the compute savings from cache reuse—sits at approximately 39% cache hit ratio.

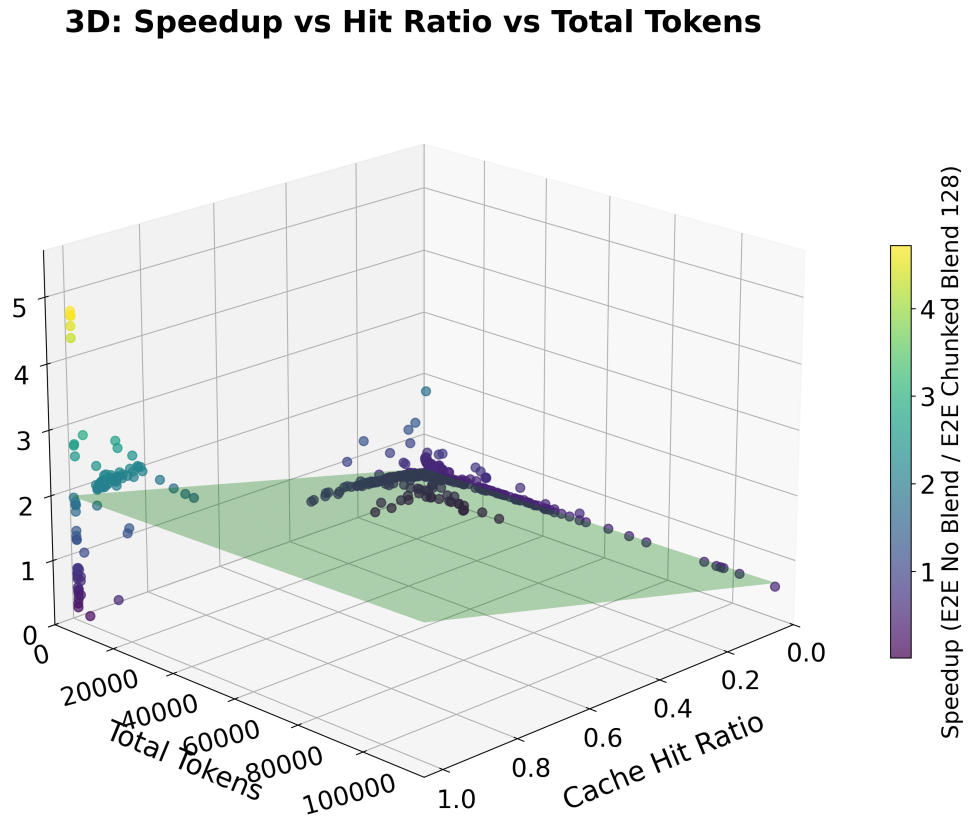

Figure 4.3: 3D surface plot of TTFT speedup vs. cache hit ratio and total prompt tokens (outliers removed). The fitted plane shows that hit ratio is the primary driver of speedup, with a breakeven point at ∼39% hit ratio.

Two distinct behavioral regimes are visible in the plot. For requests with more than 10,000 tokens, nearly all data points cluster tightly around the fitted surface, indicating that CacheBlend's speedup is highly predictable for long-context workloads. In contrast, requests below 5,000 tokens exhibit a pronounced fan-shaped spread: some achieve up to 4–5× speedup while others regress below 0.5×, despite comparable hit ratios.

Figure 4.4 breaks down the speedup distribution by token-count bucket. The <2k bucket has a mean speedup of 0.862 but an absolute range of 0.036 to 9.239, confirming that short-prompt behavior is highly unpredictable. By contrast, the 5–10k bucket shows a tighter distribution centered near breakeven (mean 0.996, with 33% of requests above 1.0×), and the >30k bucket is consistently above breakeven. This pattern further supports the observation from the 3D plot: CacheBlend's latency benefit is most reliable for the long-context prompts that are also the most expensive to recompute from scratch.

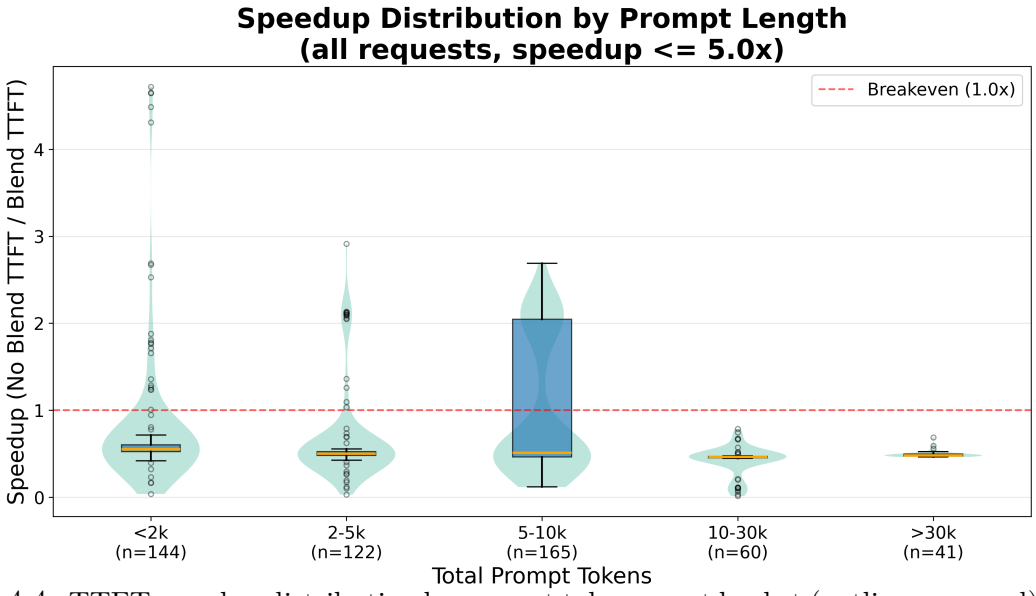

Figure 4.4: TTFT speedup distribution by prompt token-count bucket (outliers removed). Violin plots show the probability density; box plots mark the median and interquartile range.

Notably, this variance among small requests persists even when restricting to well-cached requests. Among requests with cache hit ratio $\geq 50\%$, the <2k bucket retains a spread from 0.3× to 9.5×, while the 5–10k bucket collapses to a tight 1.9–2.5× band. A high cache hit ratio alone does not guarantee fast performance for small requests; additional factors govern the outcome. We investigate these factors in detail in Section 4.2.3.

Speedup Distribution

Figure 4.5 shows the distribution of per-request TTFT speedup values across the full trace. The distribution is bimodal: the primary mode lies below 1.0× (median: 0.51×, mean: 0.77×), reflecting the overhead-dominated short-request regime discussed in Section 4.2.3. A secondary mode near 2.0× corresponds to longer-prompt requests where cache reuse outweighs the per-step overhead. While the majority of individual requests experience a net slowdown under the current implementation, the high-speedup requests tend to be the most

compute-intensive ones (exceeding 10,000 tokens), where absolute time savings are largest. It is worth noting that the SWE-bench Verified-mini trace is derived from real-world Django issues, which tend to produce shorter, more localized prompts compared to repositories with deeper dependency chains. As a result, the request length distribution skews toward the short-prompt regime where per-step overhead dominates. Workloads targeting larger or more interconnected codebases would shift the distribution toward longer prompts and higher cache hit ratios, where CacheBlend's speedup is both more consistent and more substantial.

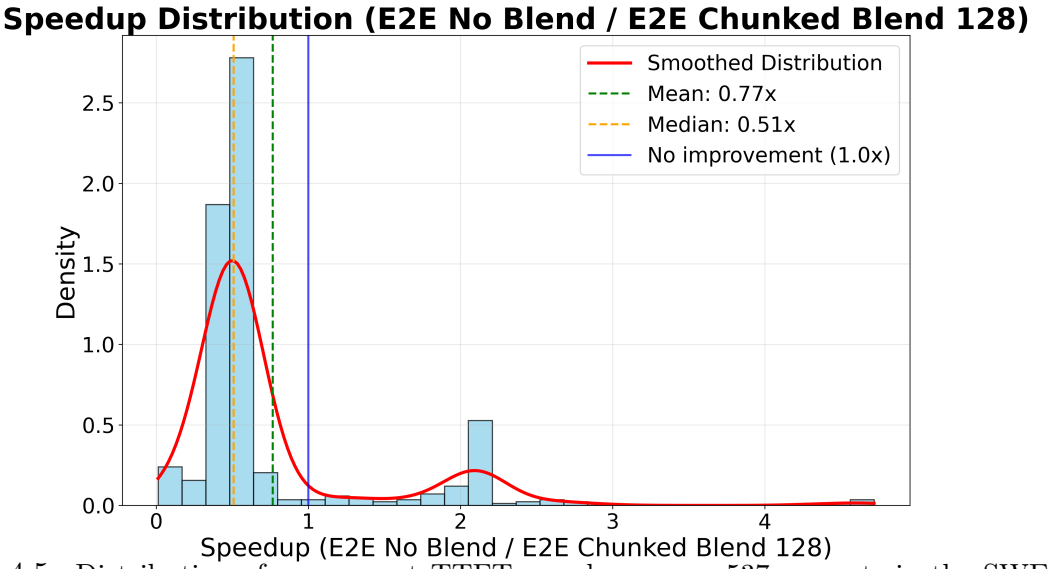

Figure 4.5: Distribution of per-request TTFT speedup across 537 requests in the SWE-bench Verified-mini trace.

4.2.3 Analysis: Small-Request Speedup Variance

The 3D speedup plot reveals that requests with fewer than 5,000 tokens exhibit significantly higher speedup variance than longer requests at comparable hit ratios. Using the per-request profiling infrastructure described in Section 3.5.4, we trace the root cause to an interaction between vLLM's chunked-prefill scheduler and LMCache's per-step invocation pattern.

Scheduler-Induced Fragmentation

vLLM's chunked-prefill scheduler determines how many tokens to process per scheduling step based on `max_num_batched_tokens` and the current batch composition. A request that arrives during a decode-heavy window is interleaved across multiple steps, each processing only a fraction of the prompt. The profiler records reveal a bimodal distribution in the number of `retrieve_layer` operations per logical request: 322 requests incur 4 operations (1 prefill step $\times$ 4 tensor-parallel workers) while 99 requests incur 36 operations (9 prefill steps $\times$ 4 workers).

Figure 4.6 cross-tabulates the fragmentation rate against prompt length. Small requests (<2,000 tokens) are fragmented at a 45% rate—3.4$\times$ the rate of 5–10k requests (10%) and

the highest of any bucket. This susceptibility arises because short prompts fill a smaller portion of `max_num_batched_tokens`, leaving room for the scheduler to pack decode steps alongside them and defer their remaining tokens to subsequent batches.

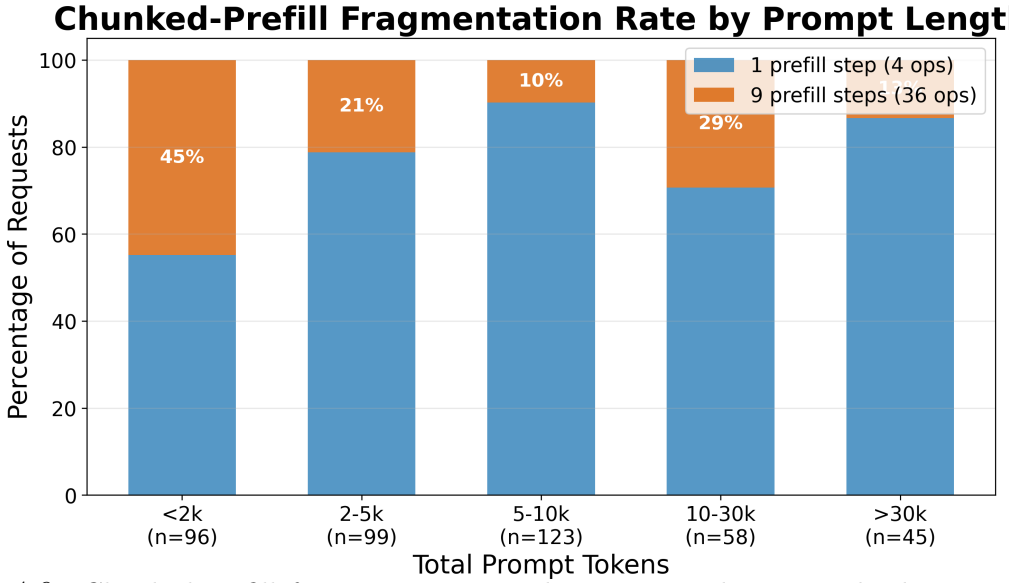

Figure 4.6: Chunked-prefill fragmentation rate by prompt token-count bucket. Requests are classified as non-fragmented (1 prefill step, 4 ops) or fragmented (9 prefill steps, 36 ops). Small requests (<2k tokens) exhibit the highest fragmentation rate at 45%.

Fixed Per-Step Overhead

Each invocation of the cache engine's retrieve path triggers a chain of operations whose cost is largely independent of the number of tokens in the step. Figure 4.7 presents the measured per-call cost broken down by phase group, aggregated over 4,852 blend and 3,092 no-blend `retrieve_layer` calls. Three phases dominate:

1. **Token database lookup** (0.7 ms blend, 0.3 ms no-blend): walks the token sequence to identify cached chunks.
2. **Storage retrieval** (2.0 ms blend, 3.1 ms no-blend): fetches each layer's KV cache from the CPU storage backend.
3. **GPU onload** (28.1 ms blend, 14.6 ms no-blend): copies KV data into vLLM's paged buffer and, in blend mode, executes gap-aware recomputation.

GPU onload accounts for 91% of blend's per-call time and exhibits a 1.93× ratio over no-blend. The ∼13.5 ms gap stems from CacheBlend's blender, which selects the top 15% most-divergent token positions per layer and recomputes their KV state to restore cross-attention accuracy. The overall mean per-call overhead is 30.8 ms in blend mode versus 18.0 ms without blending (a 1.72× ratio).

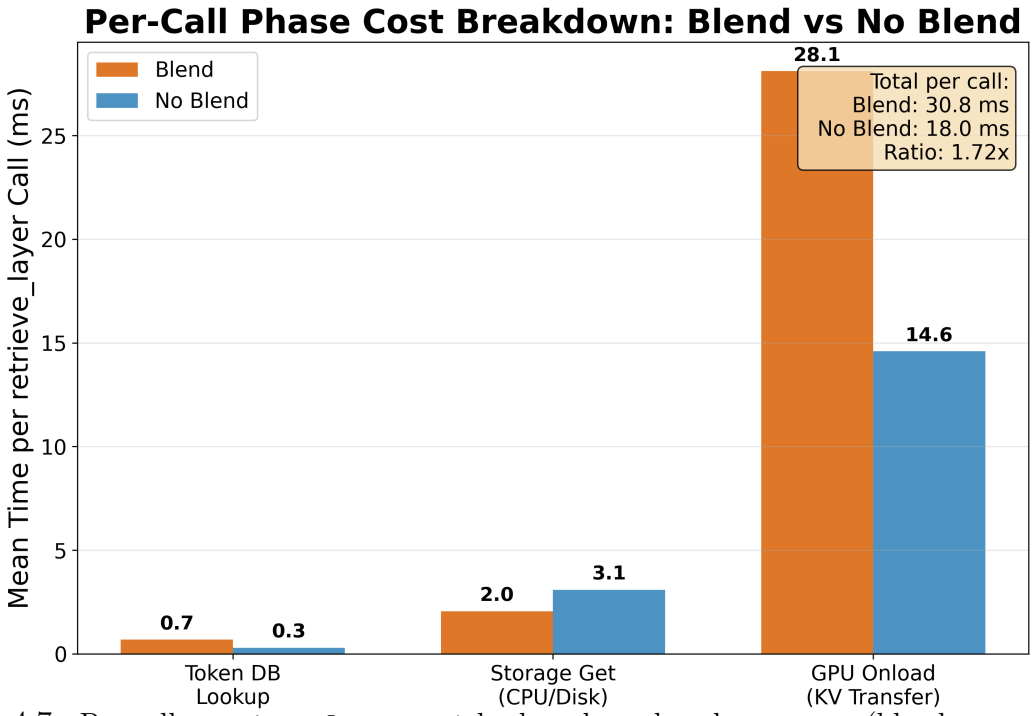

Figure 4.7: Per-call `retrieve_layer` cost broken down by phase group (blend vs. no-blend). GPU Onload dominates at 91% of blend's per-call time, with a 1.93× ratio over no-blend due to gap-aware KV recomputation.

The fixed-cost nature of this overhead creates a fundamental asymmetry with respect to prompt length. For a 1,000-token request fragmented across 9 steps with 4 tensor-parallel workers, the aggregate retrieve overhead reaches ∼1,109 ms—9.7× the ∼114 ms baseline TTFT for straight prefill of the same prompt. The compute savings from cache hits (∼100 ms) are dwarfed by this overhead. By contrast, a 50,000-token request incurs the same ∼1,109 ms of overhead, but against a ∼5,000 ms baseline (22% of total), and cache hits save ∼4,500 ms of compute, yielding a net ∼2× speedup that closely matches the fitted surface. The per-step overhead is invariant to the number of tokens in the step, but the compute savings scale linearly with token count; small requests sit below the crossover where overhead exceeds savings.

Fragmentation as the Source of Variance

The preceding analysis explains why small requests are slower *on average*, but does not fully account for the extreme *variance* observed in the <5k token regime. Figure 4.8 resolves this by plotting speedup against total tokens on a log scale, with each point colored by its fragmentation class: non-fragmented (1 prefill step, 4 ops) versus fragmented (9 prefill steps, 36 ops).

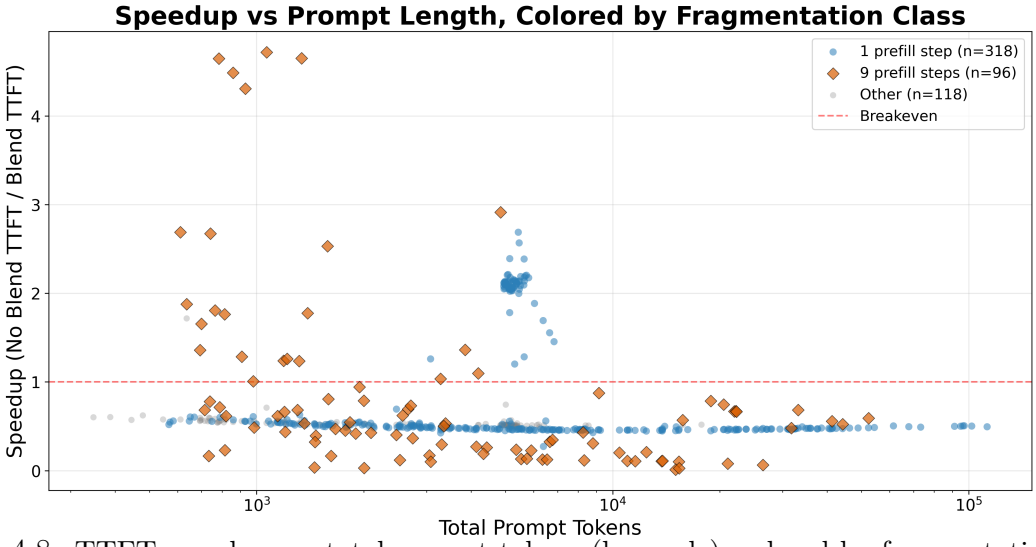

Figure 4.8: TTFT speedup vs. total prompt tokens (log scale), colored by fragmentation class. Non-fragmented requests (blue circles) cluster tightly below breakeven for small prompts; fragmented requests (orange diamonds) exhibit the full variance spread. Both classes converge for long prompts.

Two sharply distinct populations emerge among small requests:

1. **Non-fragmented small requests** (blue, 4-op) cluster tightly at ∼0.54× speedup for the <2k bucket (mean = 0.538, std = 0.031). Blend overhead is consistent and always exceeds the compute savings for these short prompts. This population is *predictable but consistently below breakeven*: each request pays a single step of 30.8 ms overhead against a baseline TTFT that is itself only ∼60–200 ms.
2. **Fragmented small requests** (orange, 36-op) exhibit the full spread from near 0× to 9.5× for the <2k bucket (mean = 1.604, std = 1.786). The variance is driven by the interaction between the number of scheduler steps and the cache hit pattern across those steps: some fragmented requests encounter favorable cache timing where cached content is warm from earlier steps in the same scheduling window, while others pay the full per-step overhead with minimal cache benefit.

The key insight is that **fragmentation is both the source of the worst underperformers and the best over-performers** among small requests. The scheduler's batching decision at request arrival time determines which fragmentation class a request falls into, and this decision depends on the instantaneous batch composition—not on the request's own cache state. Two identically sized, identically cached requests can land on opposite sides of the speedup distribution based solely on whether other decode requests happened to be in flight at the moment of arrival.

For large requests (>10k tokens), both fragmentation classes converge to the same narrow band around the fitted surface, because the per-step overhead is a small fraction of per-step compute regardless of how many steps the scheduler chose. This convergence confirms that the variance phenomenon is specific to the short-prompt regime and does not affect the primary target workload of long-context agentic prompts.

Implications

The fragmentation effect is not a fundamental limitation of CacheBlend's reuse model but rather a consequence of how the per-step invocation pattern interacts with the scheduler's batching decisions. Three mitigation strategies are under consideration:

(a) A *minimum token-per-step threshold* that skips cache retrieval for micro-batched steps, directly addressing the pathological case at the adapter level. This is the lowest-risk fix: it can be implemented in the adapter's per-request loop and tuned via the per-call overhead profile (30.8 ms blend, 18.0 ms no-blend).

(b) A *prompt-length-aware bypass* that routes sub-2,000-token requests away from the blend path entirely. The data supports this approach strongly: even non-fragmented small requests average only 0.54× speedup, indicating that blend is net-negative for short prompts regardless of fragmentation. Sub-2k requests represent ∼27% of the trace population (145/536), and their absolute compute savings are modest (∼100 ms).

(c) A *coalesced retrieval mode* that accumulates token ranges across prefill steps and issues a single batched lookup, eliminating the overhead multiplier. This is the most principled long-term solution but requires a non-trivial redesign of the adapter–cache-engine interface, as the current layerwise pipeline interleaves layer loads with the model's forward pass.

For deployments where the request length distribution is known—as in the SWE-bench trace used here—approaches (a) and (b) provide effective, low-complexity mitigations. Approach (b) is particularly appealing given the finding that even the non-fragmented population does not benefit from blend at short prompt lengths.

Bibliography

This bibliography contains 21 references.

[Ant+25] Antonis Antoniades, Albert Örwall, Kexun Zhang, Yuxi Xie, Anirudh Goyal, and William Wang. *SWE-Search: Enhancing Software Agents with Monte Carlo Tree Search and Iterative Refinement*. 2025. arXiv: 2410.20285 [cs.AI]. URL: https://arxiv.org/abs/2410.20285.

[Ant25] Anthropic. *Introducing Claude 4*. Announces Claude Code general availability. May 2025. URL: https://www.anthropic.com/news/claude-4.

[Bai+23] Ramakrishna Bairi, Atharv Sonwane, Aditya Kanade, Vageesh D C, Arun Iyer, Suresh Parthasarathy, Sriram Rajamani, B. Ashok, and Shashank Shet. *CodePlan: Repository-level Coding using LLMs and Planning*. 2023. arXiv: 2309.12499 [cs.SE]. URL: https://arxiv.org/abs/2309.12499.

[Cho+24] Neil Chowdhury et al. *Introducing SWE-bench Verified*. 2024. URL: https://openai.com/index/introducing-swe-bench-verified/.

[Gim+24] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. *Prompt Cache: Modular Attention Reuse for Low-Latency Inference*. 2024. arXiv: 2311.04934 [cs.CL]. URL: https://arxiv.org/abs/2311.04934.

[Ho+20] Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. *Constructing A Multi-hop QA Dataset for Comprehensive Evaluation of Reasoning Steps*. 2020. arXiv: 2011.01060 [cs.CL]. URL: https://arxiv.org/abs/2011.01060.

[Jia+25] Zhonghao Jiang, Xiaoxue Ren, Meng Yan, Wei Jiang, Yong Li, and Zhongxin Liu. *CoSIL: Software Issue Localization via LLM-Driven Code Repository Graph Searching*. 2025. arXiv: 2503.22424 [cs.SE]. URL: https://arxiv.org/abs/2503.22424.

[Kwo+23] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. 2023. arXiv: 2309.06180 [cs.LG]. URL: https://arxiv.org/abs/2309.06180.

[LIB25] Lukas, Ian, and Balta. *Cursor Agent CLI*. https://cursor.com/en/blog/cli. Cursor Blog. Aug. 2025.

[Mar25] MariusHobbhahn. *SWE-bench-verified-mini*. 2025. URL: https://huggingface.co/datasets/MariusHobbhahn/swe-bench-verified-mini.

[MS25] Taylor Mullen and Ryan J. Salva. *Introducing Gemini CLI: your open-source AI agent*. Google Developers Blog. 2025.

[Ouy+25] Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia, Jiawei Han, Hongming Zhang, and Dong Yu. *RepoGraph: Enhancing AI Software Engineering with Repository-level Code Graph*. 2025. arXiv: 2410.14684 [cs.SE]. URL: https://arxiv.org/abs/2410.14684.

[SKS25] Nick Sharma, Mahi Kolla, and Ilai Soloducho. *Fully Reimagined: AI-First Google Colab*. Google Developers Blog. May 2025. URL: https://developers.googleblog.com/en/fully-reimagined-ai-first-google-colab/.

[Tri+22] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. *MuSiQue: Multihop Questions via Single-hop Question Composition*. 2022. arXiv: 2108.00573 [cs.CL]. URL: https://arxiv.org/abs/2108.00573.

[Xia+24] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. *Agentless: Demystifying LLM-based Software Engineering Agents*. 2024. arXiv: 2407.01489 [cs.SE]. URL: https://arxiv.org/abs/2407.01489.

[Yan+24] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. 2024. arXiv: 2405.15793 [cs.SE]. URL: https://arxiv.org/abs/2405.15793.

[Yao+25] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. *CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion*. 2025. arXiv: 2405.16444 [cs.LG]. URL: https://arxiv.org/abs/2405.16444.

[Zha+24a] Lei Zhang, Yunshui Li, Jiaming Li, Xiaobo Xia, Jiaxi Yang, Run Luo, Minzheng Wang, Longze Chen, Junhao Liu, and Min Yang. *Hierarchical Context Pruning: Optimizing Real-World Code Completion with Repository-Level Pretrained Code LLMs*. 2024. arXiv: 2406.18294 [cs.CL]. URL: https://arxiv.org/abs/2406.18294.

[Zha+24b] Xinrong Zhang et al. *∞Bench: Extending Long Context Evaluation Beyond 100K Tokens*. 2024. arXiv: 2402.13718 [cs.CL]. URL: https://arxiv.org/abs/2402.13718.

[Zha+24c] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. *AutoCodeRover: Autonomous Program Improvement*. 2024. arXiv: 2404.05427 [cs.SE]. URL: https://arxiv.org/abs/2404.05427.

[Zhe+24] Lianmin Zheng et al. *SGLang: Efficient Execution of Structured Language Model Programs*. 2024. arXiv: 2312.07104 [cs.AI]. URL: https://arxiv.org/abs/2312.07104.